

Superpowers obra/superpowers

CRITICAL — Install this ~

- What it actually is

The single most genuinely useful tool in this list. 42,000+ stars. In the official Anthropic Claude plugin marketplace. Built by Jesse Vincent (Prime Radiant). Not a collection of prompts — it is a complete development methodology enforced as a skills system that activates automatically.

- What it forces the model to do

1. Brainstorm and ask clarifying questions BEFORE writing any code. 2. Write a detailed implementation plan broken into 2–5 minute tasks. 3. Follow strict TDD: failing test must exist before implementation. 4. Code-review each task against the plan before moving to the next. 5. Verify all tests pass before considering work done. The model cannot skip these steps — they are enforced, not suggested.

Auto-triggers TDD enforced Plan before code Built-in code review

- The 3 commands you will actually use

/superpowers:brainstorm — before starting any new feature or layer. Forces Socratic dialogue, surfaces edge cases, produces design before code. /superpowers:write-plan — converts the brainstorm into a concrete task list, each task small enough to verify. /superpowers:execute-plan — runs the plan with review checkpoints between each task. Claude can work autonomously for hours without drifting from the plan.

- How it maps to your Naam Jap build

Before Layer 2 (targetEngine.js): run /superpowers:brainstorm with the formula spec. Before Layer 4 Dashboard: run /superpowers:write-plan. This turns your 6-layer build plan into verified, checkpointed execution rather than hoping each layer stays coherent. The code-reviewer agent catches logic drift between your data layer and your UI layer automatically.

- Install command

/plugin marketplace add obra/superpowers-marketplace then /plugin install superpowers@superpowers-marketplace — takes 30 seconds in Antigravity/Claude Code.

The core insight of Superpowers: the biggest problem with AI coding is not intelligence — it is discipline. The model knows how to code. It does not naturally know when to stop and think. Superpowers enforces the stop.

ClaudeMem thedotmack/claude-mem

USEFUL — Add it ~

- What it actually is

Persistent memory across sessions for Claude. Stores facts, decisions, preferences, and context in a structured file that gets injected at the start of every new conversation. Solves the "explain everything from scratch every session" problem.

- Why it matters for your project specifically

Your app has deep context: Radhavallabh sampraday rules, the target formula, the 6-layer architecture, 5 themes, the specific visual constraints, the no-Om/no-Garuda rule. Without memory, every new Antigravity session starts blank and you re-explain everything. With ClaudeMem, the model loads your project context automatically — it already knows your sampraday, your formula, your constraints.

Project context Design decisions Sampraday rules Architecture state

- What to store in it for your app

The target formula. The 5 theme names and their color tokens. The 6 section names. The visual forbidden list (no Om, no Garuda, no standard shadows, etc.). The current build layer and what is done vs in progress. Key architectural decisions made. This becomes your project memory that never needs to be re-explained.

Limitation: it is only as good as what you put in it. You need to actively maintain it as decisions are made. Set a habit: at the end of every productive session, update the memory file with what was decided and what is next.

Awesome Claude Code

hesreallyhim/awesome-claude-code

REFERENCE — Not a tool ~

- What it actually is

A curated list of tools, agents, plugins, and workflows — a directory, not a framework. Think of it as an index. You do not install it or use it directly. You browse it when you are looking for a specific type of tool and want to know what options exist.

- When to use it

When you hit a specific problem — "I need a tool that does X" — and want to know if something already exists in the Claude ecosystem. Not something to read front to back. Bookmark it, search it when you have a specific need.

- For your app specifically

Low direct value right now. You already have the tools you need. Check it after Phase 4 if you hit a specific gap — for example, if you need a tool to help with Capacitor Android packaging or Google Drive API integration.

Value: 3/10 for your current situation. High value in general as a reference. Do not spend time on it now.

UI/UX Pro Max

nextlevelbuilder/ui-ux-pro-max-skill

SOLID — Use alongside @frontend-design ~

- What it actually is

A SKILL.md file — a markdown instruction set that gets loaded into Claude before UI work. It enforces professional design system thinking: visual hierarchy, spacing systems, typography scale, component states (default, hover, active, disabled, error), accessibility, and motion principles. More opinionated than the Vercel @frontend-design skill.

- The difference from @frontend-design

@frontend-design (Vercel official) focuses on React best practices and code quality. UI/UX Pro Max focuses more on design decisions — spacing, hierarchy, states, interaction quality. They are complementary, not competing. Use both: @frontend-design for component architecture, UI/UX Pro Max for visual design quality.

Component states

Visual hierarchy

Motion principles

Spacing systems

- How to load it

Place the SKILL.md file in your .agent/skills/ folder. Then in Antigravity before any UI work: "Use @ui-ux-pro-max to build the GlassCard component." The skill content prepends to Claude's context for that task, enforcing design quality before a single line of CSS is written.

Honest caveat: skills like this are only as good as their content. Read it once before using it. If any of its default design principles conflict with your specific aesthetic (which they might — it is built for generic premium apps, not devotional Vrindavan aesthetics), note the conflicts and add your overrides explicitly in your prompt.

Security Checklist

Vibe Coding Security Prompts

RELEVANT — 3 of 7 apply to you ~

Reality check first: this checklist is written for web apps with user accounts, public APIs, and external traffic. Your app is a personal Android APK with no server, no user accounts, no public endpoints. Most of these prompts address threats that do not exist in your threat model.

- Check 1 — Leaked Secrets: LOW PRIORITY

Your app has one external service: Google Drive API. The OAuth token is handled by Capacitor's native plugin — it never lives in your JS code. There are no hardcoded keys to scan for. Run this check once before APK build, but it is not a primary concern.

- Check 2 — Input Sanitization: NOT APPLICABLE

You have no database queries from user input. You have no SQL. Your inputs are numbers (jap counts) and text (notes). IndexedDB stores them locally. There is no injection vector in a local-only personal app.

- Check 3 — Rate Limiting: NOT APPLICABLE

You have no public API endpoints. No external AI model calls. This check is irrelevant for your architecture.

- Check 4 — Auth Architecture: NOT APPLICABLE

No login. No user accounts. No authentication layer. Intentional by design.

- Check 7 — Dependency Check: HIGH PRIORITY ✓

This one is genuinely relevant. Before your APK build, run: "Review my package.json. Identify any libraries with known vulnerabilities, unmaintained packages, or redundant dependencies." React, Vite, Three.js, and Capacitor all have active ecosystems — but third-party plugins (especially Capacitor community plugins) can have security issues. Run this once before building the APK.

- The Global System Prompt: ADAPT AND USE ✓

The suggested global prompt is excellent — but adapt it for your context. Remove the "prefer managed services" and "custom authentication" parts. Add: "Never store sensitive data unencrypted in localStorage. Keep the Google Drive OAuth token in the native Capacitor secure storage, never in JS-accessible storage. Treat the quote library as read-only data — never allow user input to modify it programmatically."

- Data Integrity Check: ADD THIS — not in their list

The check that actually matters for your app is not in this list: data integrity. Your target formula creates a chain — if one record gets corrupted, every future target is wrong. Add this prompt to your own checklist: "Audit the DailyRecord write operations. Ensure every write is atomic. Ensure the recomputation chain cannot be interrupted mid-execution. Ensure IndexedDB transactions roll back on failure rather than leaving partial state."

Specific to your app More important than XSS

Summary: use the global system prompt (adapted), the dependency check, and the data integrity check. Skip the other 5 — they address a different threat model entirely.

Your Complete Toolkit

Final integrated stack — everything in one place

Ready to use ~

- Always loaded (persistent)

ClaudeMem — project context, formula, constraints, current layer. Your adapted global security system prompt — data integrity focus. 5 theme tokens in memory.

- Before any new layer or complex feature

/superpowers:brainstorm — surfaces edge cases, forces design before code. /superpowers:write-plan — breaks work into verifiable 2–5 minute tasks.

- Before any UI component

@frontend-design (Vercel) + @ui-ux-pro-max — both loaded. Then: state your forbidden list explicitly before asking for the component.

- During execution

/superpowers:execute-plan — checkpointed execution with automatic code review between tasks.

- When stuck on 3D

Switch to ChatGPT 120B with the specific Three.js problem. Get the working code. Bring it back to Sonnet for React integration. Never stay stuck — route to the specialist.

- After each layer completes

Gemini Pro High for cross-review. Feed the full layer code + screenshots. Update ClaudeMem with what was completed and what decisions were made.

- Before APK build

Dependency check prompt. Data integrity audit. Google Drive token storage verification.

This is the complete professional studio setup. Install Superpowers now — it will change how every remaining layer gets built. Everything else slots in as needed.